DETECTION OF TRAFFIC SIGNS USING CONVOLUTIONAL NEURAL NETWORKS

Abstract

This paper presents a development of a machine learning program for Traffic Sign Recognition using Convolutional Neural Network applied to the German Traffic Sign Recognition Benchmark dataset. The dataset contains over 43 classes with a total of more than 50,000 images. My approach uses CNN architecture to capture complex image features, through the different convolution layers, pooling layers, fully connected layers to predict the output class probabilities for the 43 different classes in the dataset. To address the underrepresentation of certain classes in the model, the number of images in the classes underrepresented has been increased by doubling them, this adjustment to the dataset creates a more balanced dataset and increases the performance of the model across the 43 different classes. The results show that the model provides a foundation for automated road transportation.

Introduction

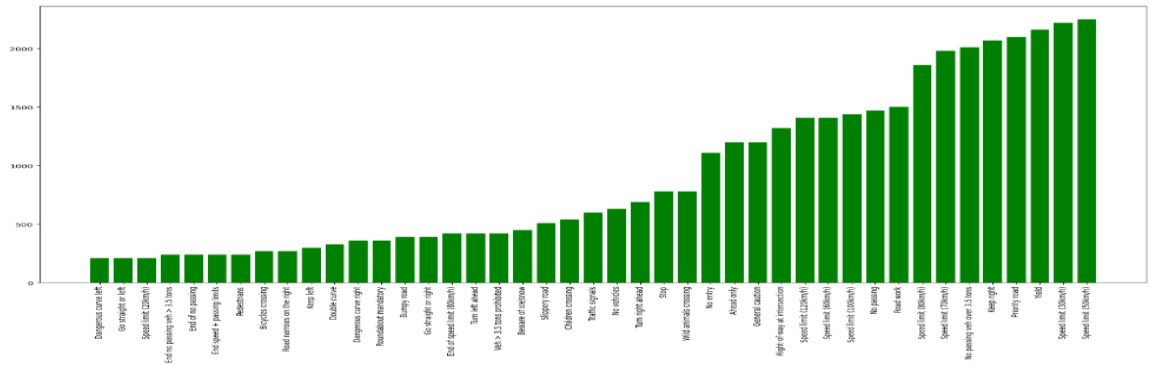
Image Recognition in today's world has made a huge leap in the developments in machine learning and deep learning techniques with the developments of AlexNET, Residual Neural Networks, Convolutional Neural Networks etc.

Convolutions Neural Networks has a played a big role in enhancing the performance of image recognition, CNN are helpful in learning complex features of an image making them the best for complex categorical image classification such as object detection etc. This is having proven to be better than manual feature extraction

Giving the increase in demand for smarter and safer transportation vehicles, this paper presents a model for traffic sign detection an important component for these self-driving vehicles. Using CNN architecture, the model predicts traffic signs under various conditions.

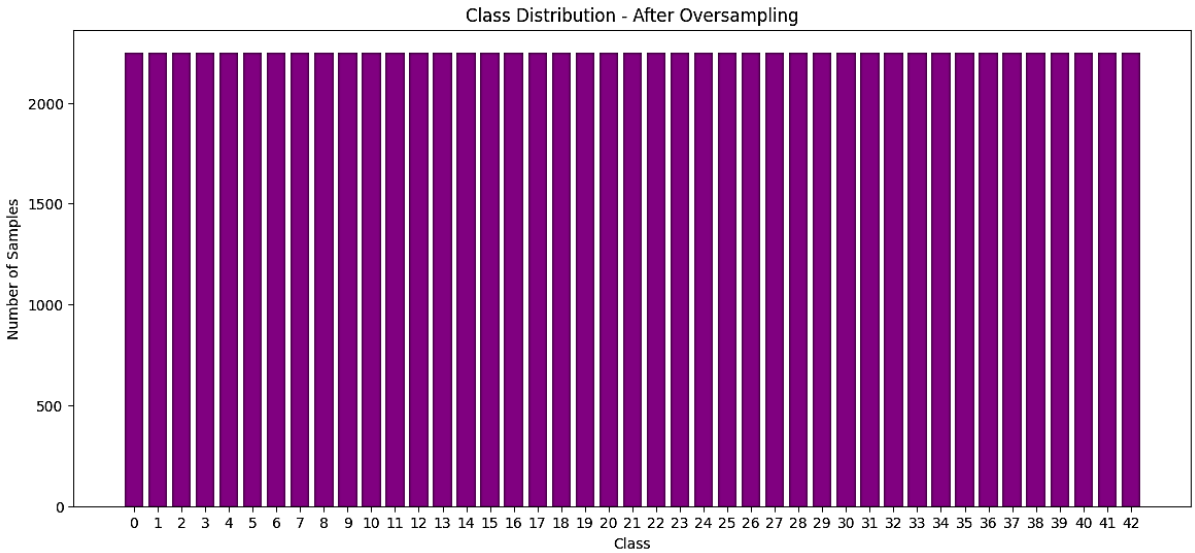
Data Analysis

The dataset used by the model is the GTSRB (The German Sign Recognition Benchmark), consist of training and testing data with over 50,000 images. The training data has around 39,000 images split across 43 classes, with the testing data having more than 12,000 images.



Through the above graphical representation that the dataset is imbalanced and some traffic signs are under sampled. For example, the keep right, keep left, go straight, or left these classes are under sampled which could lead to misclassification of the signs if the model learn from the classes with a greater number of samples during the training process.

Due to the undersampling of certain classes observed, to solve the issue two approaches were considered either down sampling the overrepresented classes or oversampling the underrepresented samples to create a more balanced dataset. In this model, the second approach is taken into consideration where the class with the highest number of images used as a reference and the undersampled classes have been matched by increasing their representation.



To make sure that the model does not overfit to the data and learn unwanted patterns which might have been present in the dataset, the dataset and its respective labels are shuffled.

Convolution Neural Network Architecture

1. Input Layer
   The model takes RGB images as the input where the resolution of each image is 30x30 pixels. The input shape for the model would be 30x30x3, 3 here represent RGB channels (red, green, blue).

2. First Convolutional Layer
   The first convolutional layer uses 32 filters of size 3x3. The filters with the default strides help extract the features from the images. The Rectified Linear Unit (ReLU) is applied to the layer to introduce non linearity, this helps the model understand the complex patters.

3. Second Convolutional Layer
   The second convolutional layer consist of 64 filters of size 3x3, as the image size decreases from layer to layer, with the filters increasing the model can understand more complex patters. Like the first Convolutional layer Rectified Linear Function used as the Activation function.

4. First Pooling Layer
   After the second convolutional layer, a maxpooling layer is introduced with a pool size of 2x2 is applied on the output of the second convolutional layer, maxpooling reduces the size of the image by taking the maximum value present in the 2x2 matrix, this layer helps the model retain the most important features of the image with decreasing computational power.

After the pooling layer BatchNormalization is applied to improve the training process by reducing internal covariate shifts, which is changing the weights in earlier layers causes slowness in the ability for the mode to learn, to address this BatchNormalization is considered and to help the model converge faster and make the training process more stable in lower number of iterations

5. Third Convolutional Layer
   The third convolutional layer has 128 filters of size 3x3, during this layer the image is very small and the layer can extract more detailed features and other unique patterns in the image, this layer uses Rectified Linear Function to introduce non linearity like the first and second convolutional layers.

6. Fourth Convolutional Layer
   The fourth Convolutional Layer has 256 filters of size 3x3, during this layer which is the last convolutional layer of the model, the 256 filters slide across the image and extract all the minimal details left after passing through the 3 convolutional layers before, same as the 3 convolutional layers we use Rectified Linear Function to introduce Non linearity.

7. Second Pooling Layer
   We introduced another pooling layer after the third and fourth convolutional layers to help extract the most important details after the image has passed through all the 4 convolutional layers, same as the first one we use maxpooling with a pooling size of 2x2, this pooling layer extracts all the finer details of the image before passing to the fully connected layers
   We add BatchNormalization to make sure the model can converge faster passing through all these layers.

8. GlobalAveragePooling
   In the model to make sure that the model 2D features are converted to a 1D vector format instead of using flatten, GlobalAveragePooling2D is used. This layer helps the model reduce the spatial dimensions while still preserving the channels, as GlobalAveragePooling reduces the number of parameters by reducing the image to a 1D vector format it helps remove the risk of overfitting by not passing many parameters to the Dense/Fully connected Layers, when using this it averages out the spatial dimensions and only keep the important features of the image and not the noise around the features. Helping the model generalize well.

9. First Fully Connected Layer
   Dense layer or a fully connected layer connects every neuron in one layer to every neuron to the previous layer, allowing the nodes to learn from the previous layer. 512 neurons are present in the first Dense layer with the Rectified Linear Unit as the activation function to introduce non-linearity in the model, the Dense layer takes inputs from the previous layers add weights and biases and pass through the activation function which is then passed through to the next layer
   BatchNormalization used to address Internal Covariate shift and help in the convergence of the model during training.
   Dropout layer which is a regularization method in deep neural networks which helps overfitting, it sets some neurons in the connected layers to 0 during training in the model the dropout rate has been set to 0.5 meaning the 50% of the neurons are set to zero during training.

10. Second Fully Connected Layer
    In the second Fully connected layers there is 256 neurons present, with Rectified Linear unit as the activation function.
    BatchNormalization used again to help with internal covariate shift.
    Dropout used like previous layer but only dropout 30% of the neurons than the 50% in the first Dense layer, meaning only 30% percent of the neurons are made 0 during the training process.

11. Third Fully Connect Layer

In the third Fully connected layer neurons are decreased to 128, with Rectified Linear Unit as the activation function for this layer, as noticed the neurons keep decreasing each layer this helps the model in feature extraction as in the earlier dense layer only low level features and edges and textures are learnt, by minimizing the neurons at each connected layer the models ability to feature extract increases and is able to build a more complex representation , this also helps in better generalization.

BatchNormalization is added to address internal Covariate shift and help converge as model nears to the output layer.

A dropout layer is also added same as the second fully connected layer drops out 30% of the neurons during training helping the model not depend on neurons with higher weights always.

12. Output Layer

In the output layer which is also a fully connected layer has 43 neurons indicated the 43 classes present in the dataset, each output is a predicted probability of the 43 classes.

The output layer used softmax as the activation function, as this function is best for multiclass classification problems, the output is transformed into probabilities that sum up to 1

After the softmax each class is given a probability between 0 and 1 which is the highest probability is the predicted class.

Layer (type)	Output Shape	Param #
conv2d_4 (Conv2D)	(None, 28, 28, 32)	896
conv2d_5 (Conv2D)	(None, 26, 26, 64)	18,496
max_pooling2d_2 (MaxPooling2D)	(None, 13, 13, 64)	0
batch_normalization_4 (BatchNormalization)	(None, 13, 13, 64)	256
conv2d_6 (Conv2D)	(None, 11, 11, 128)	73,856
conv2d_7 (Conv2D)	(None, 9, 9, 256)	295,168
max_pooling2d_3 (MaxPooling2D)	(None, 4, 4, 256)	0
batch_normalization_5 (BatchNormalization)	(None, 4, 4, 256)	1,024
global_average_pooling2d_1 (GlobalAveragePooling2D)	(None, 256)	0
dense_3 (Dense)	(None, 512)	131,584
batch_normalization_6 (BatchNormalization)	(None, 512)	2,048
dropout_2 (Dropout)	(None, 512)	0
dense_4 (Dense)	(None, 256)	131,328
batch_normalization_7 (BatchNormalization)	(None, 256)	1,024
dropout_3 (Dropout)	(None, 256)	0
dense_5 (Dense)	(None, 128)	32,896
batch_normalization_8 (BatchNormalization)	(None, 128)	512
dropout_4 (Dropout)	(None, 128)	0
dense_6 (Dense)	(None, 43)	5,547

Training and Testing the model

1) Training

For training the dataset has been split into training and validation sets, to ensure the model can evaluate its performance on data it has not seen before, helps to make sure that the model is not memorizing the training data.

To update the model's parameters to minimize loss Adam optimizer used which is a parameter which has best features of SGD, AdaGrad and RMSProp, its best to train deep neural networks.

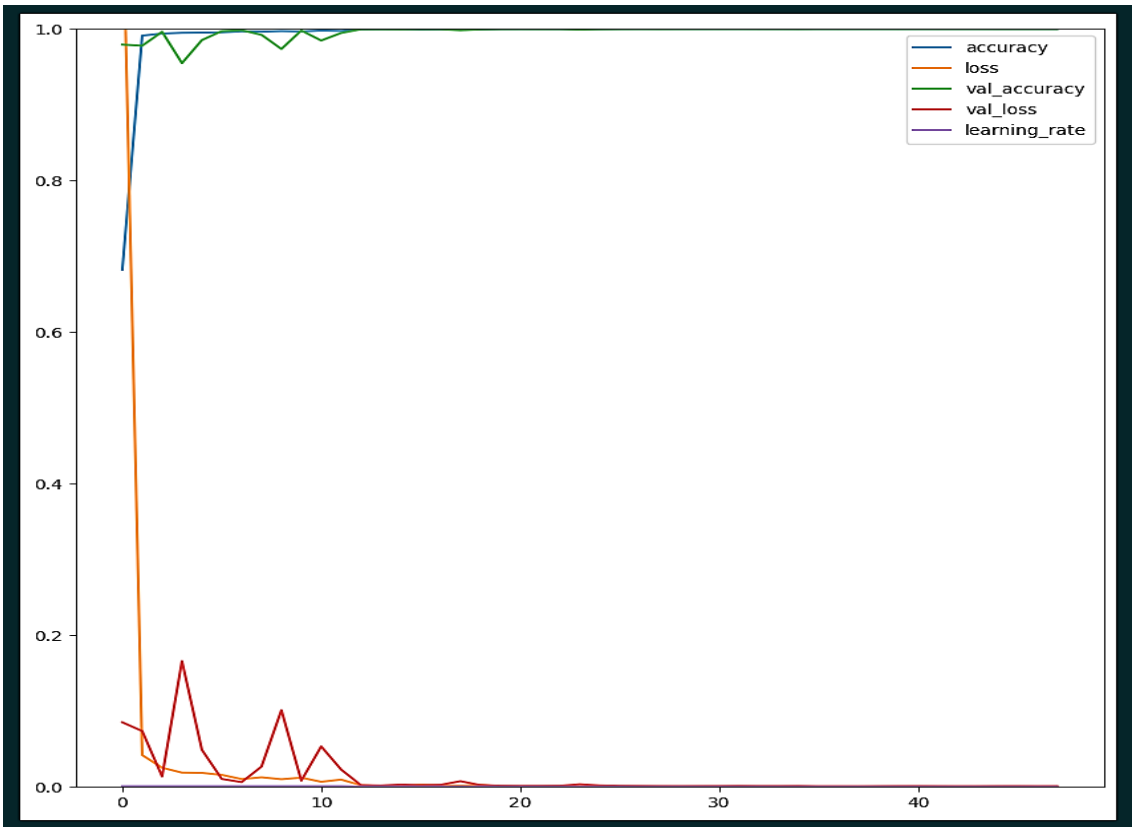
Hyperparameters such as learning rate is used to help the model control the steps to increase the weights and biases on each iteration to minimize the loss function, the learning rate in the model is set to 0.001

The loss function used in the model is categorical crossentropy as the model data is one-hot encoded

To reduce overfitting and overtraining the model uses early stopping and reduceLROnPlateau, early stopping is used to help the model from overfitting by keeping an eye on the validation loss, when the validation loss has no improvement, the training is halted.

ReduceLROnPlateau is another way to help reduce overfitting by reducing the learning rate when then the validation loss is not improving. The lower learning rates can help model converge better and generalize better and helps the model from overfitting or memorizing the training data

.



From the above graph we can see the model is able to train well by attaining local minima by the lower epochs with batch size of 84, this shows the model is able to train well and can predict well which is seen by the validation accuracy and validation loss metric.

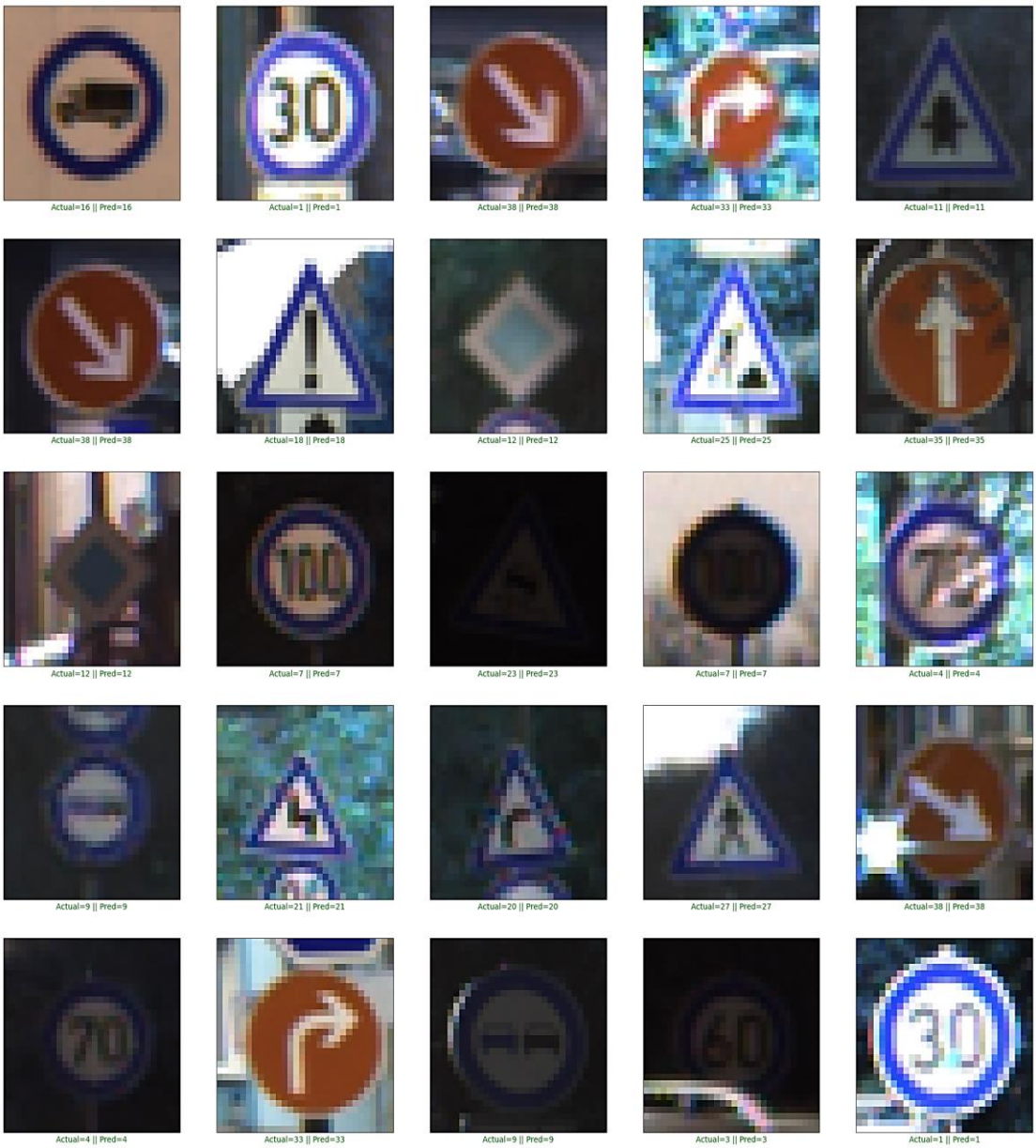
2) Testing

The model's performance on testing dataset gives an accuracy of 98.749, suggesting that the model is doing well overall

Other performance metrics such as precision and recall scores also have been calculated as these metrics are important when it comes to the autonomous vehicles

The precision score for the model 0.980 when the model predicts a right class its correct 98% of the time, which is very high precision rate

The recall score for the model is 0.979 when the model correctly understands true positives which shows that the model has low rate of false negatives.



The model still cannot detect 1.3% of the images from the testing dataset, on a deeper dive on to why this occurred we see that the most of the images that were not detected where distorted or the images where not recognisable by the naked eye. Ex: Signs having shadows, Signs covered by tree leaves, branches, sun gleaming on the signs making it unrecognizable and unpredictable.

Conclusion

Through the outcomes, the model can demonstrate that Convolutional Neural Networks can be used for traffic-sign recognition, which is the first step for autonomous vehicles. Through a better well-rounded dataset and other deep-learning techniques and other model architectures the precision and the accuracy of the model can be increased. The performance metrics accuracy, precision and recall indicate a positive approach has been taken as the performance metrics are high, also shows that Convolution Neural Network on its own is sufficient to predict under minimal condition

While the model yields good results, there are still some grey areas where the model could perform better under different variations, as seen where the models lack prediction when weather or other conditions are introduced to the model.

References

Dataset: https://www.kaggle.com/datasets/meowmeowmeowmeowmeow/gtsrb-german-traffic-sign

Image recognition and pattern recognition: https://www.educative.io/